Clínica Virtual

Sistema de Gestión de Turnos Médicos — Guía de flujos, arquitectura y organización del código

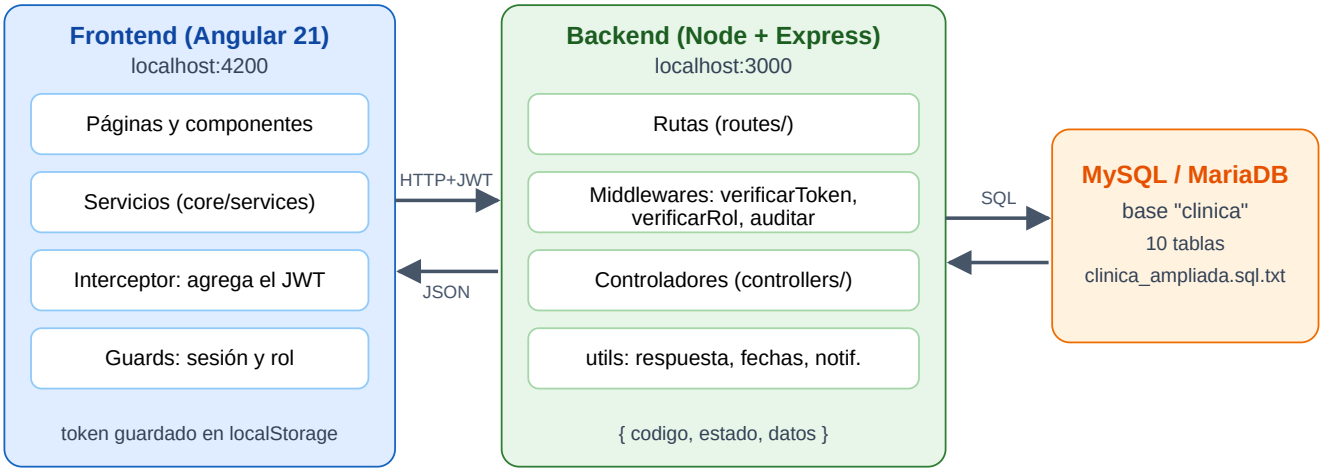
Trabajo práctico integrador · Programación 2 · Entrega final del frontend

Contenido

1. Cómo funciona el sistema (arquitectura)
2. Roles y qué puede hacer cada uno
3. Flujos comunes: inicio, login, registro, notificaciones y seguridad
4. Flujos del paciente
5. Flujos del médico
6. Flujos del operador
7. Flujos del administrador
8. Flujo de punta a punta: la vida de un turno
9. Reglas de negocio y validaciones
10. Organización de carpetas (frontend y backend)
11. Cómo levantar el sistema y usuarios de prueba

1. Cómo funciona el sistema

El sistema tiene tres piezas que corren por separado y se comunican entre sí:



Pieza	Tecnología	Responsabilidad
Frontend	Angular 21 + Angular Material + Chart.js	Lo que ve el usuario. Muestra pantallas según el rol, valida formularios antes de enviar y llama a la API.
Backend (API REST)	Node.js + Express + JWT + bcrypt	Recibe las peticiones, verifica el token y el rol, aplica las reglas de negocio y consulta la base.
Base de datos	MySQL / MariaDB	Guarda usuarios, sedes, especialidades, coberturas, agenda, turnos, historial, notificaciones y auditoría.

Recorrido de una petición (ejemplo: el médico marca un turno como atendido)

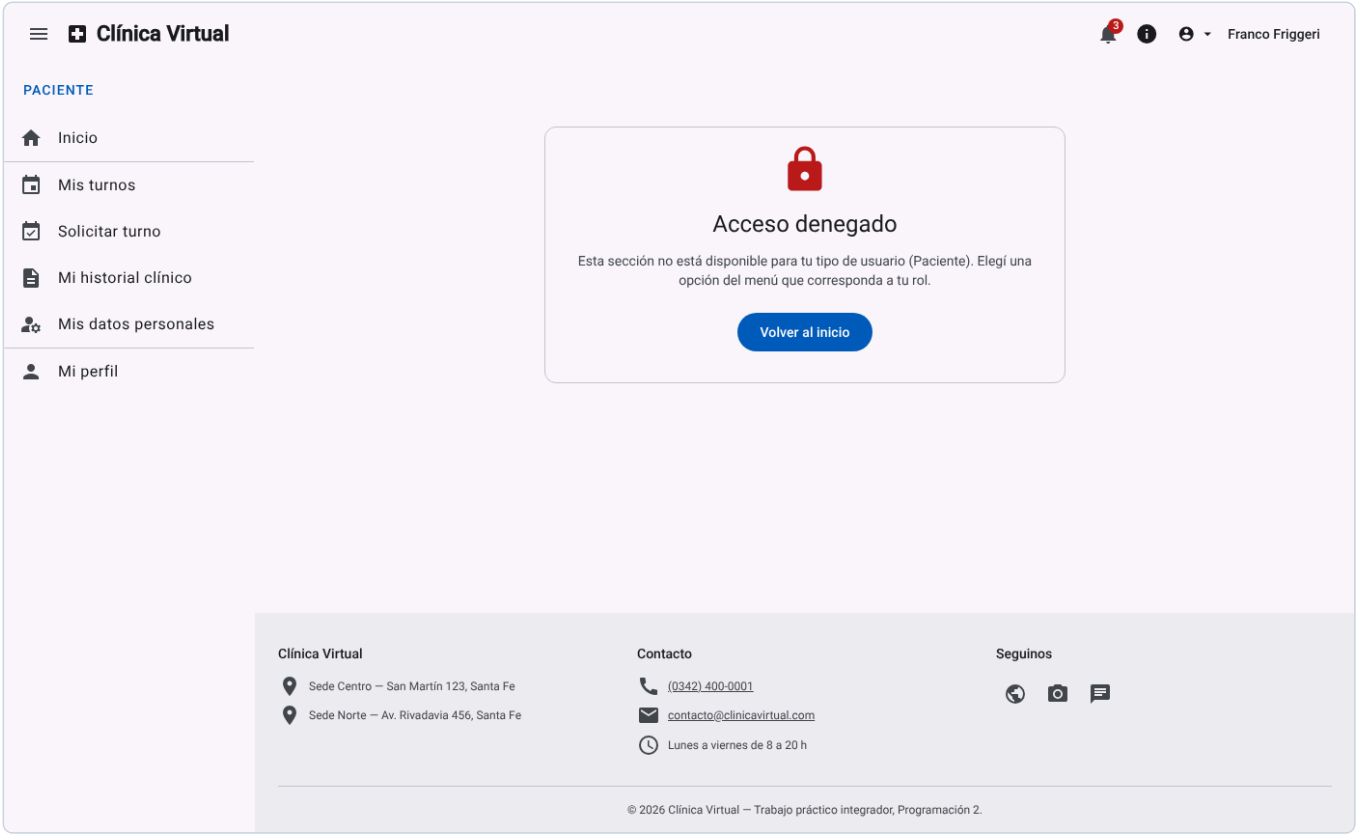
1. El médico completa el diálogo **Marcar como atendido** (componente `AtenderDialogo`). El formulario valida que el diagnóstico no esté vacío.
2. El componente llama a `TurnoService.atender(id, datos)`. El servicio arma la URL a partir de `API_URL` (definida una sola vez en `environments/`).
3. El **interceptor** `tokenInterceptor` agrega la cabecera `Authorization: Bearer <token>`.
4. En el backend, la ruta `PUT /turnos/:id/atender` pasa por `verificarToken` (401 si no hay token o venció) y `verificarRol('medico')` (403 si el rol no corresponde).
5. El controlador `turnoController.atender` verifica que el turno sea del médico y que esté confirmado; en una transacción lo marca como atendido, guarda el historial clínico y crea las notificaciones del paciente y del médico.
6. Responde con la estructura uniforme `{ codigo: 200, estado: "ok", datos: {...} }`.
7. El frontend muestra un aviso de éxito, recarga la tabla y refresca las notificaciones: el contador de la campana sube sin recargar la página.

> **Errores:** si algo falla, el backend responde con la misma estructura y el mensaje en `estado` (por ejemplo `409` "El turno no se puede atender porque su estado actual es cancelado"). El frontend lo muestra tal cual. Si el backend está apagado, se muestra "No se pudo conectar con el servidor" con un botón Reintentar.

2. Roles y qué puede hacer cada uno

Rol	Menú	Puede	No puede
Paciente	Mis turnos · Solicitar turno · Mi historial clínico · Mis datos personales	Pedir turnos, ver y cancelar los suyos, ver su historial, editar email, teléfono, cobertura y contraseña	Ver datos de otros pacientes, editar su nombre/DNI/fecha de nacimiento
Médico	Mi agenda · Turnos programados · Historial clínico	Gestionar su agenda en cualquier sede, ver/atender/cancelar sus turnos, ver el historial de lo que él atendió	Tocar la agenda o turnos de otro médico, ver historial que no atendió
Operador	Agenda y turnos · Alta de paciente · Asignar turno	Cargar agenda a cualquier médico en su sede, cancelar turnos de su sede, dar de alta pacientes, asignar turnos en su sede	Modificar agenda o turnos de otras sedes (solo consulta)
Administrador	Sedes · Especialidades · Coberturas · Usuarios · Auditoría · Reportes	ABM de catálogos, alta/edición de usuarios, ver auditoría y reportes	Gestionar agenda o turnos (no es su función)

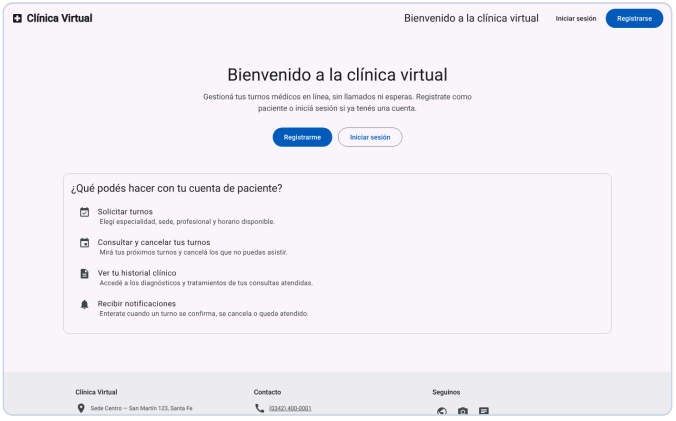
Cada sección está protegida dos veces: en el **frontend** con `sesionGuard` (sin sesión → login) y `rolGuard` (otro rol → "Acceso denegado"), aunque se escriba la URL a mano; y en el **backend** con `verificarToken` y `verificarRol`, que es la protección real.



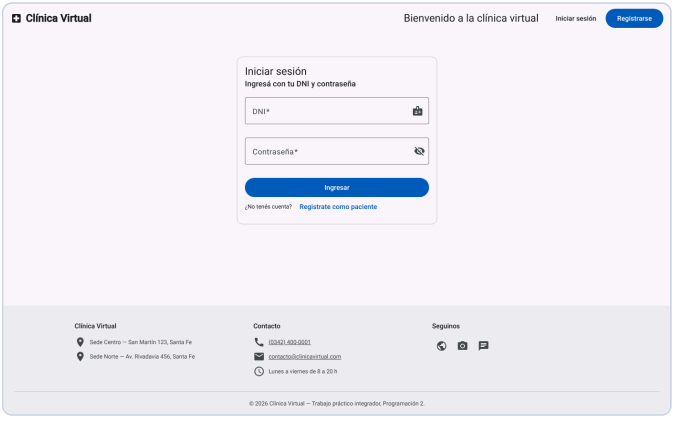
Un paciente escribe a mano /admin/sedes: el rolGuard lo lleva a "Acceso denegado".

3. Flujos comunes

Inicio, login y registro

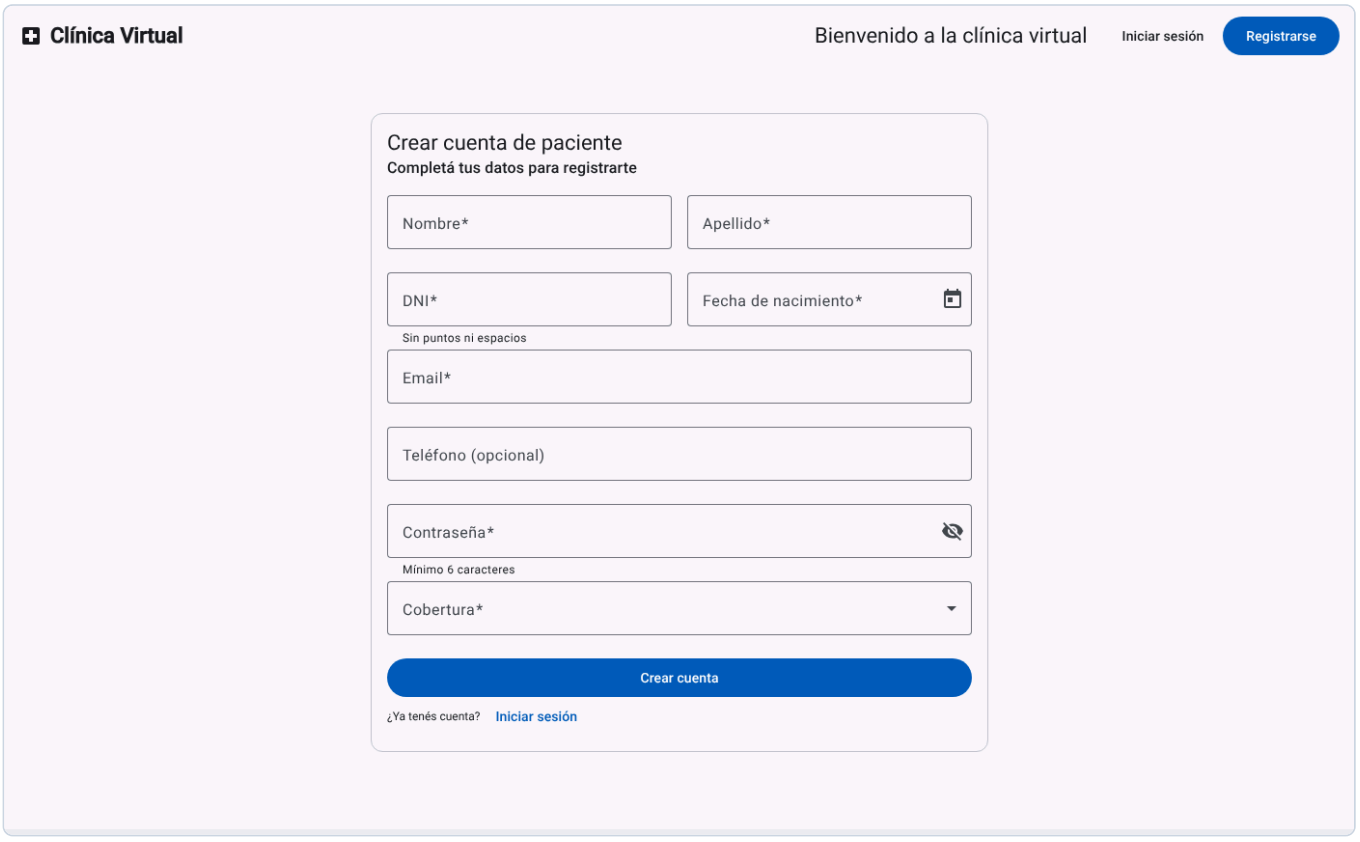


Pantalla de inicio sin sesión



Inicio de sesión con DNI y contraseña

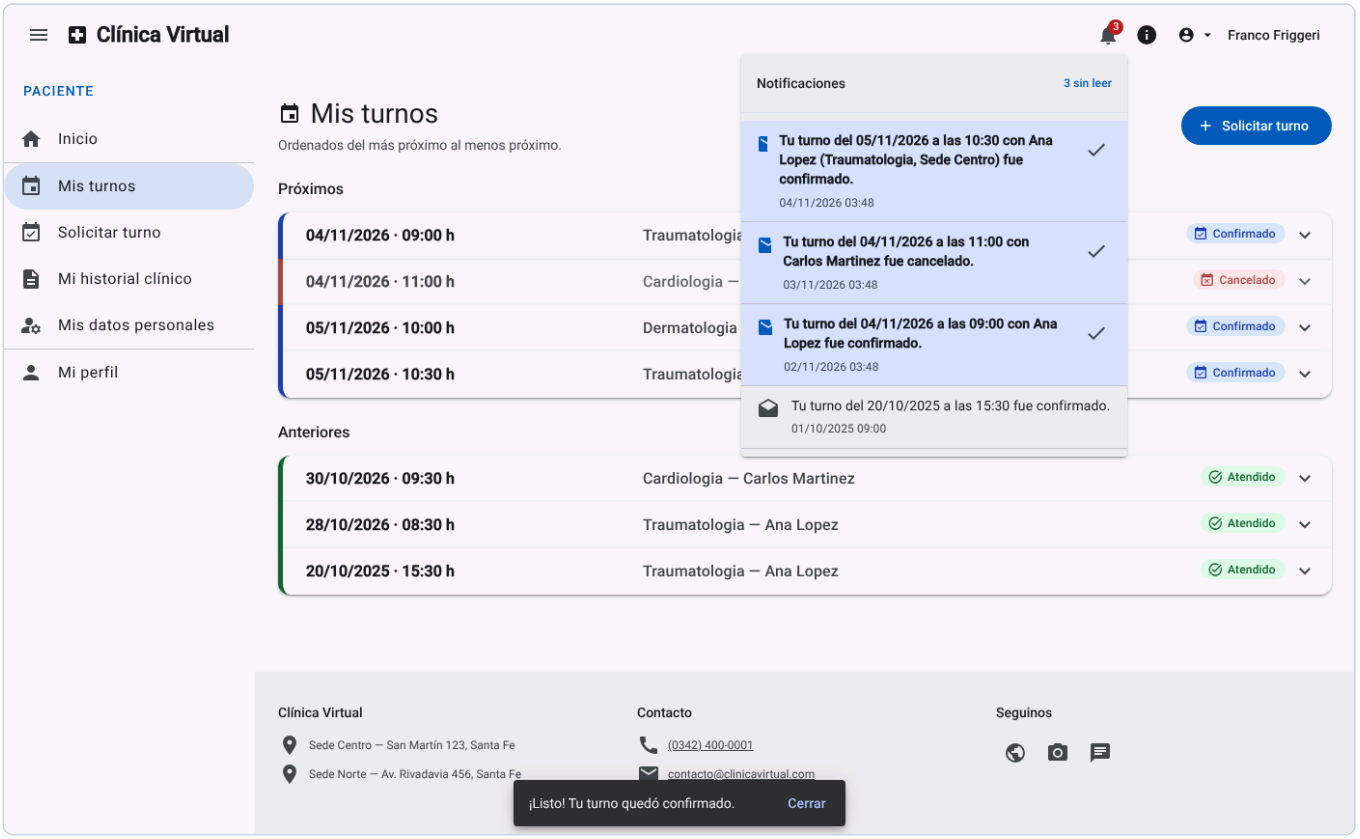
1. El usuario ingresa DNI y contraseña. `POST /auth/login` devuelve el token (con id, rol e id_sede).
2. El token se guarda en `localStorage` y enseguida se pide `GET /auth/perfil` para tener nombre, sede y cobertura.
3. Se cargan las notificaciones y se programa el cierre automático de sesión para cuando venza el token.
4. El menú lateral muestra solo las opciones del rol (`layout/menu/opciones-menu.ts`).



Registro público de pacientes. El mismo formulario (FormularioPaciente) lo usa el operador.

> **Endpoints que usa:** `POST /auth/login` · `GET /auth/perfil` · `POST /auth/registro` · `GET /coberturas`

Notificaciones (todos los roles)



La campana muestra la cantidad de no leídas; las no leídas se resaltan y tienen el botón ✓ para marcarlas.

1. Se piden al iniciar sesión, cada vez que el usuario navega y al abrir la campana (`GET /notificaciones`).
2. Si no hay no leídas, el número no se muestra.
3. Al marcar una como leída (`PUT /notificaciones/:id/leida`) el contador baja al instante.
4. El backend genera notificaciones automáticamente al **confirmar**, **cancelar** o **atender** un turno: al paciente, al médico y a quien hizo la acción (por ejemplo el operador). Quien hace la acción ve el contador actualizado sin recargar; el otro usuario lo ve la próxima vez que entra o navega.

Sesión vencida y backend caído

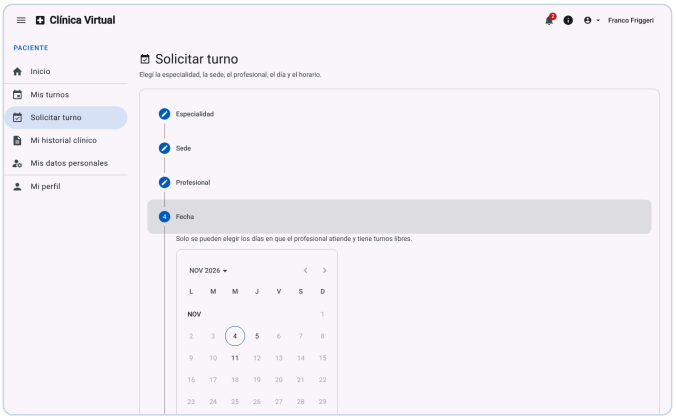
Si el token vence, el interceptor detecta el `401` (o el temporizador del vencimiento se dispara), se cierra la sesión y se vuelve al login con el aviso "sesión vencida". Si el backend no responde, cada pantalla muestra un mensaje de error con "Reintentar" y nunca queda en blanco.

4. Flujos del paciente

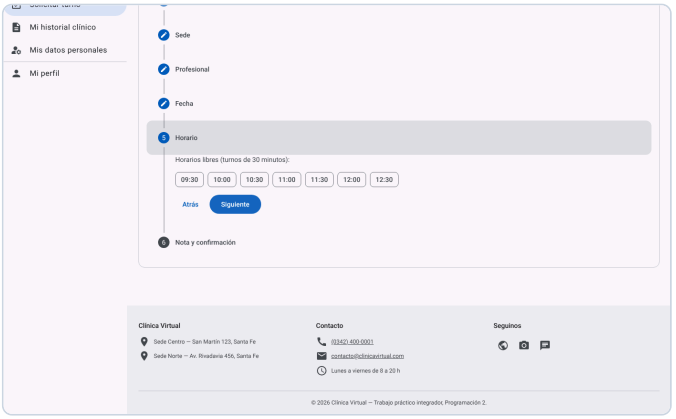
4.1 Solicitar turno (paso a paso)

Es un asistente de 6 pasos (`shared/components/solicitud-turno`). Cada paso filtra las opciones del siguiente y, si se cambia un paso anterior, se vacían los siguientes.

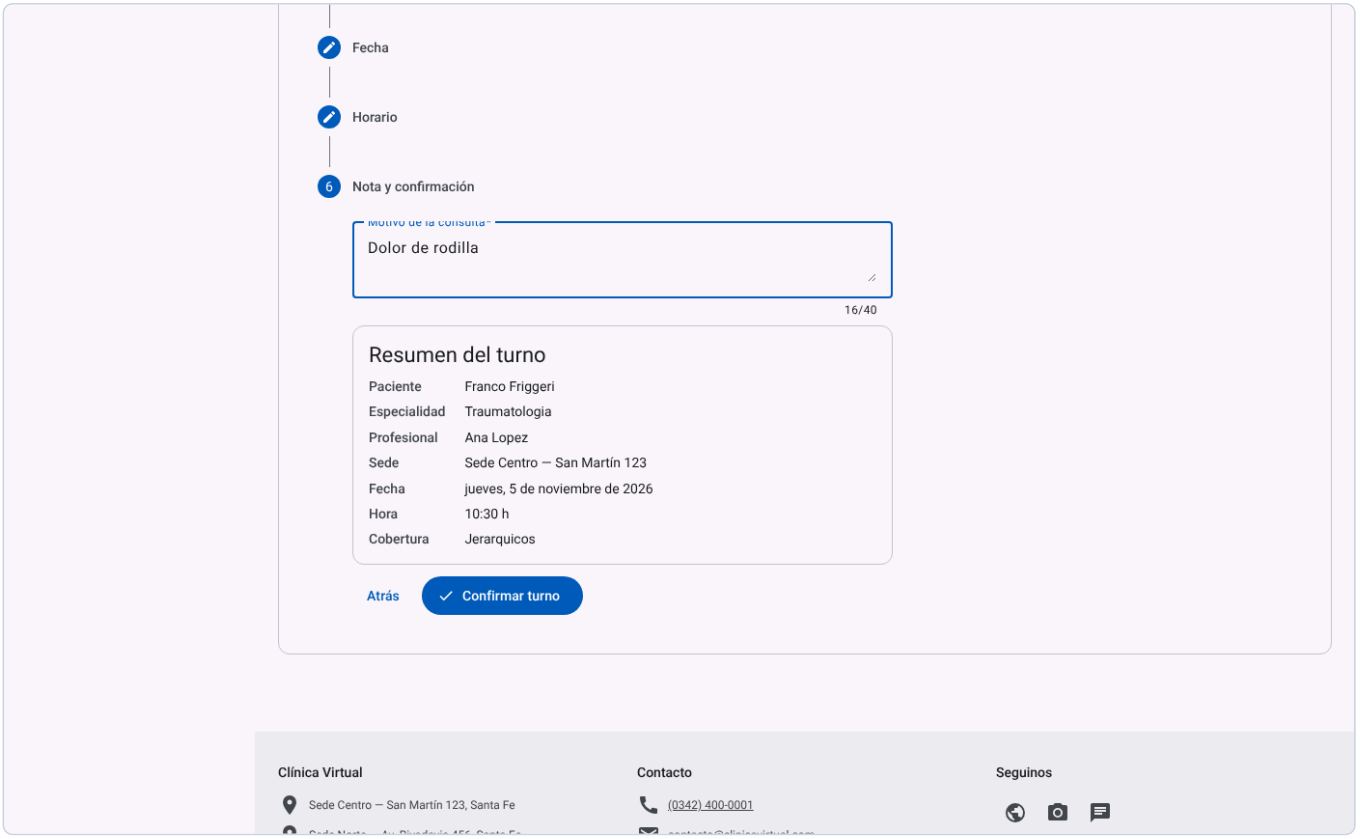
Paso	Qué se elige	De dónde salen las opciones
1	Especialidad	`GET /especialidades`
2	Sede	`GET /sedes` (para el operador queda fija en su sede)
3	Profesional	`GET /medicos?id_especialidad&id_sede&disponibles=1` : solo médicos con agenda en esa sede y especialidad
4	Fecha	`GET /disponibilidad/fechas` : el calendario solo habilita los días con agenda y lugar
5	Horario	`GET /disponibilidad/horarios` : turnos de 30 min de la agenda, menos los ya reservados y los ya pasados
6	Nota + confirmación	Resumen con especialidad, médico, sede, fecha, hora y la **cobertura registrada** del paciente (no se elige). `POST /turnos`



Paso 4: solo se pueden elegir los días con agenda



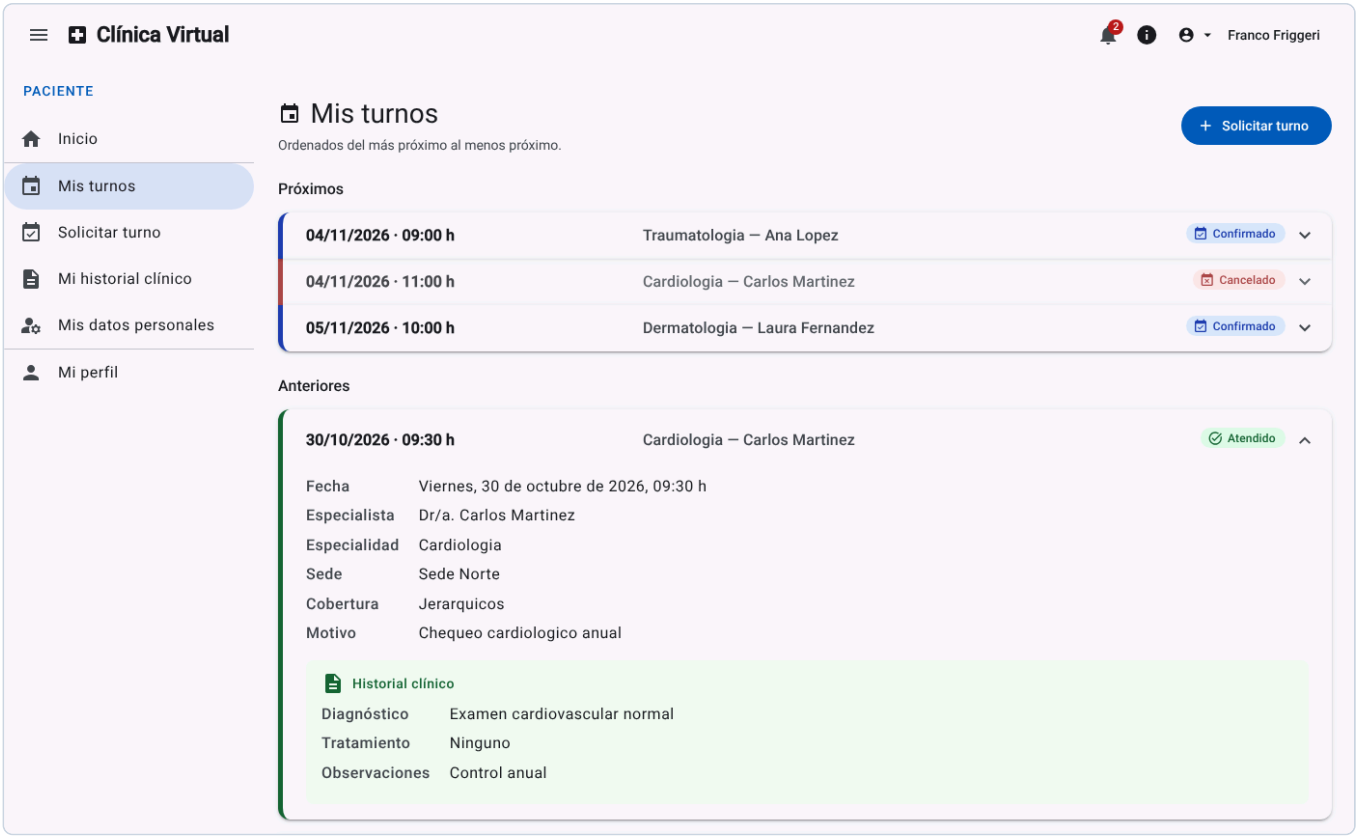
Paso 5: horarios libres de ese día



Paso 6: nota obligatoria (máx. 40 caracteres) y resumen antes de confirmar

> **Si el horario se ocupó mientras tanto**, el backend responde 409; se muestra el mensaje, se recargan los horarios y el asistente vuelve al paso 5 para elegir otro.

4.2 Mis turnos

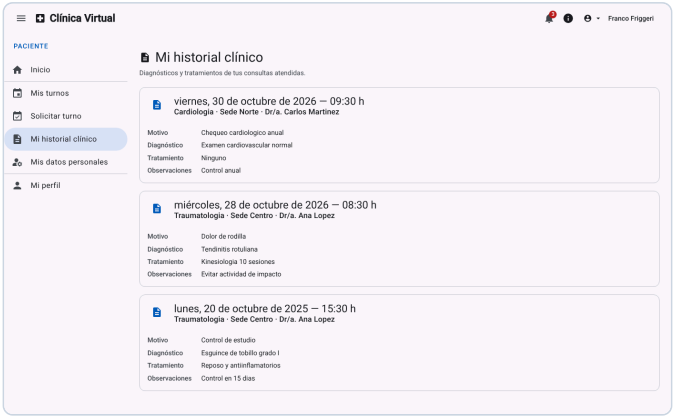


Próximos (del más cercano al más lejano) y anteriores. El turno atendido muestra el historial clínico.

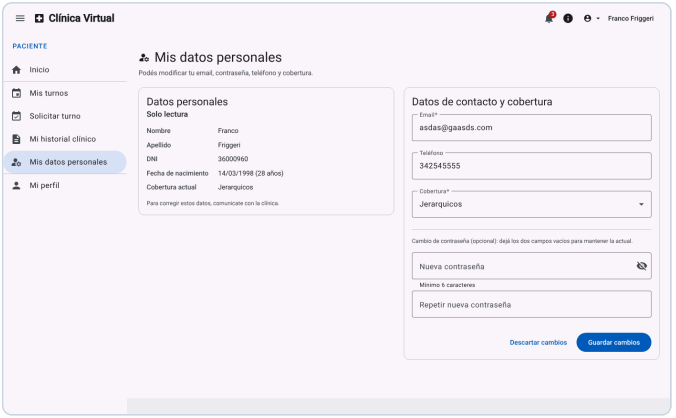
1. Se listan con `GET /turnos/mios`, separados en Próximos y Anteriores.

2. Cada estado tiene su color: azul confirmado, verde atendido, rojo cancelado.
3. Al desplegar se ve la fecha completa, especialista, especialidad, sede, cobertura y motivo.
4. Los confirmados tienen "Cancelar turno" (con confirmación) → `PUT /turnos/:id/cancelar`.
5. Los atendidos muestran diagnóstico, tratamiento y observaciones.

4.3 Mi historial clínico y Mis datos personales



Historial clínico completo del paciente



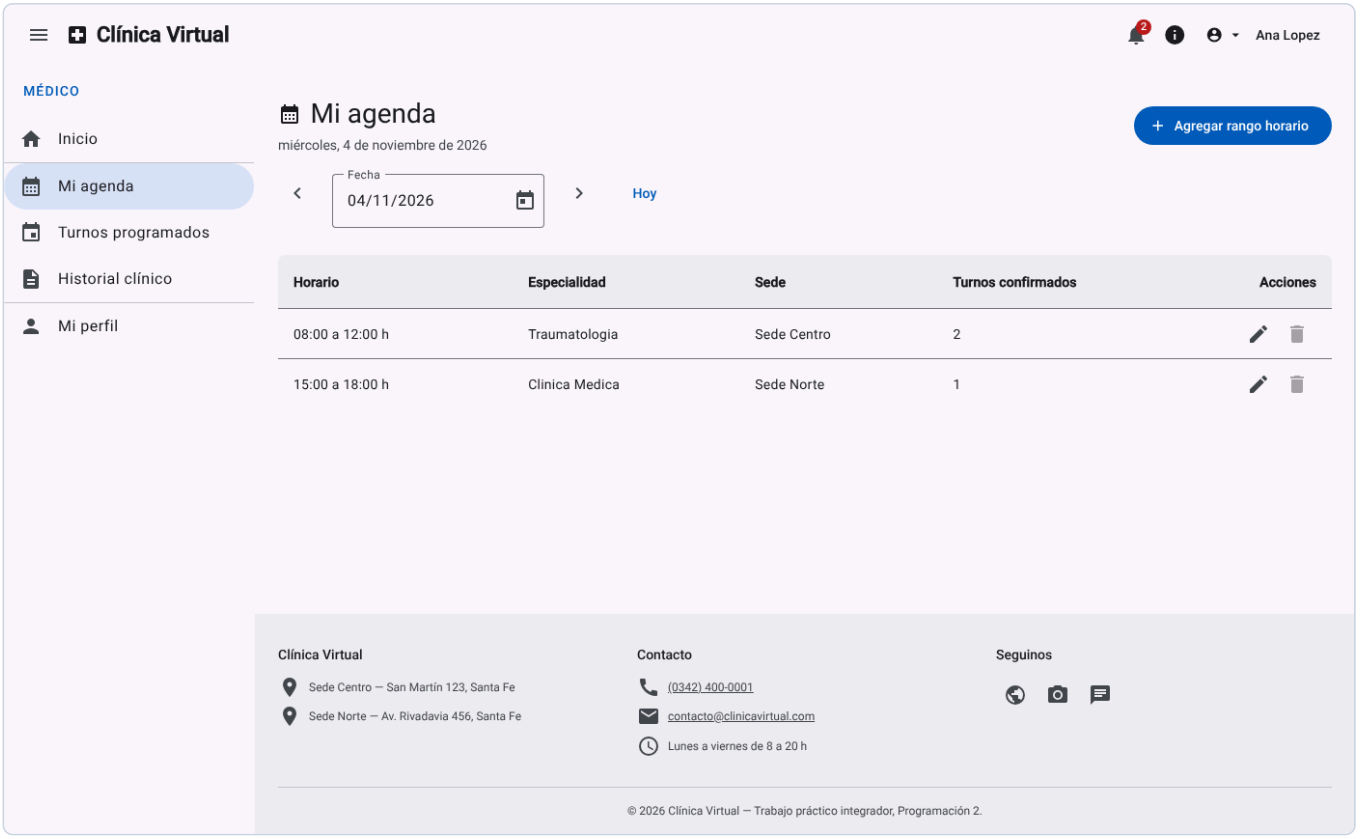
Datos personales: solo se editan email, teléfono, cobertura y contraseña

> **Endpoints que usa:** `GET /historial/:idPaciente` · `PUT /auth/perfil` · `GET /coberturas`

El cambio de contraseña es opcional y pide repetirla; el formulario no deja enviar si no coinciden. El backend ignora cualquier otro campo que se intente enviar.

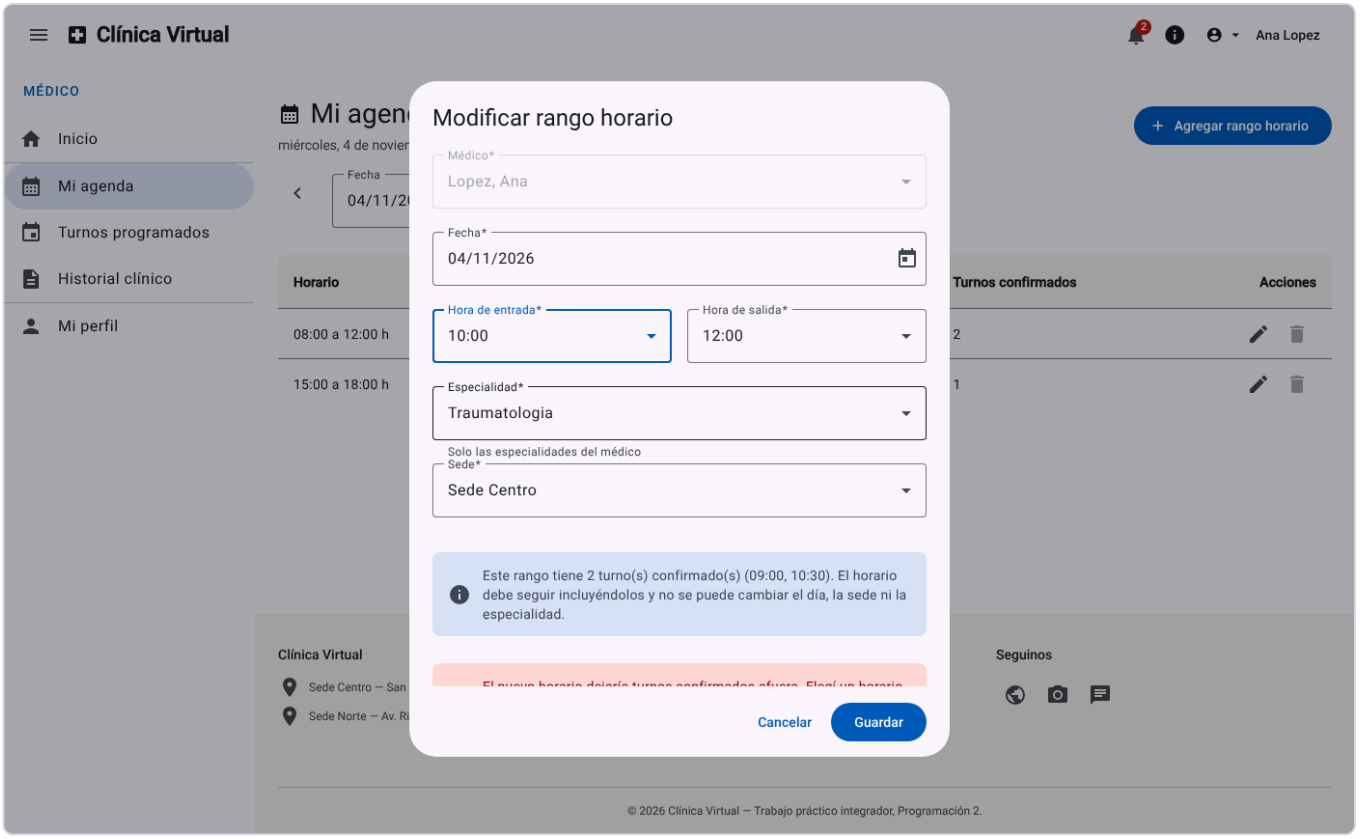
5. Flujos del médico

5.1 Mi agenda



Agenda del día (por defecto hoy). Ana López atiende en dos sedes el mismo día. El botón eliminar está deshabilitado porque los rangos tienen turnos.

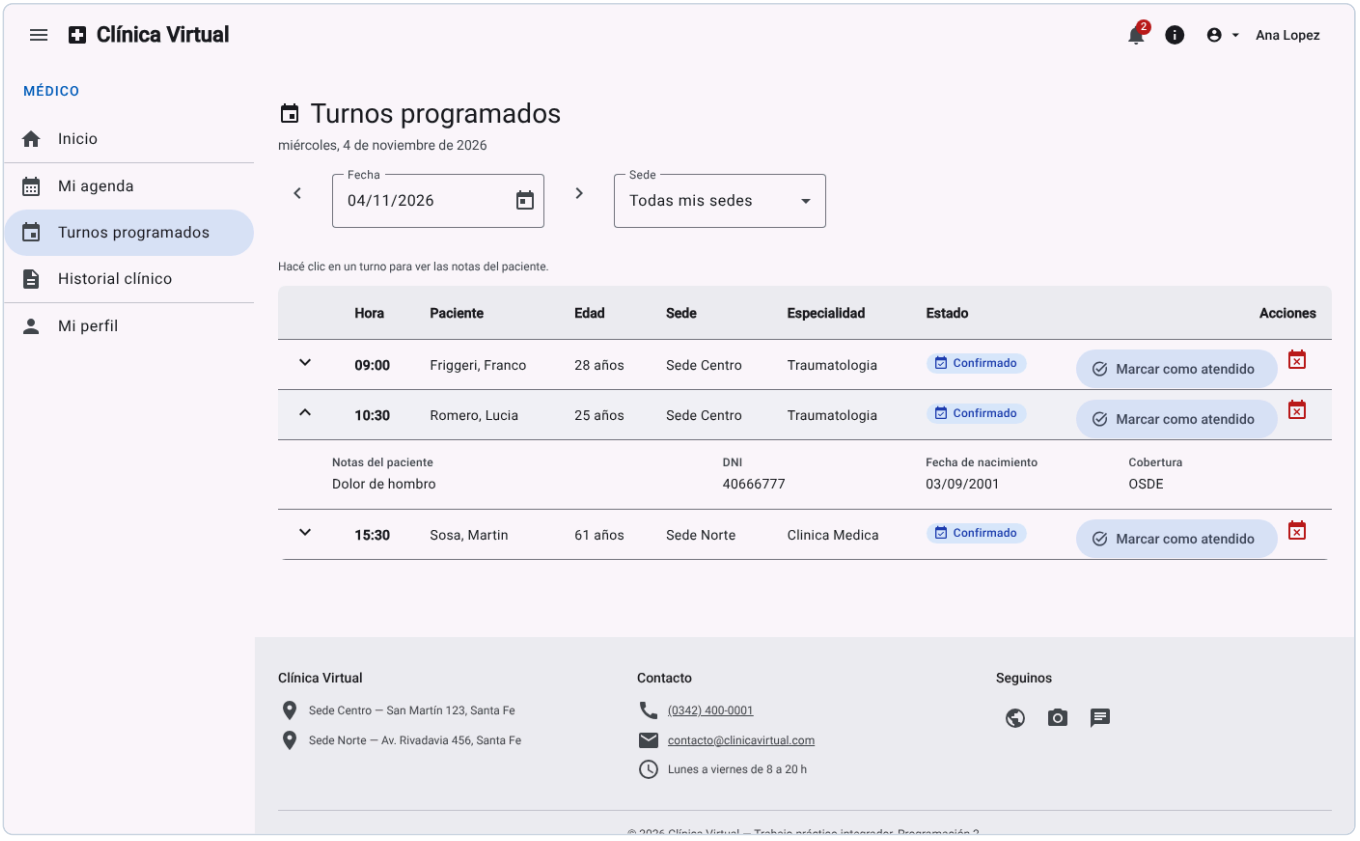
1. Se elige el día con el selector o las flechas; se listan sus rangos con `GET /agenda?fecha=` (el backend fuerza que sean los propios).
2. "Agregar rango horario": fecha, hora de entrada y salida (cada 30 min), especialidad (solo las del médico) y sede (con la suya preseleccionada). Puede cargar varios rangos el mismo día.
3. Modificar: si el rango tiene turnos confirmados, el formulario avisa antes de enviar si el nuevo horario los dejaría afuera, o si se cambia día, sede o especialidad.
4. Eliminar: solo si no tiene turnos confirmados, con confirmación previa.



Al intentar empezar a las 10:00 un rango que tiene un turno a las 09:00, la interfaz lo impide y lo explica.

> **Endpoints que usa:** `GET /agenda` · `POST /agenda` · `PUT /agenda/:id` · `DELETE /agenda/:id` · `GET /medicos` · `GET /sedes`

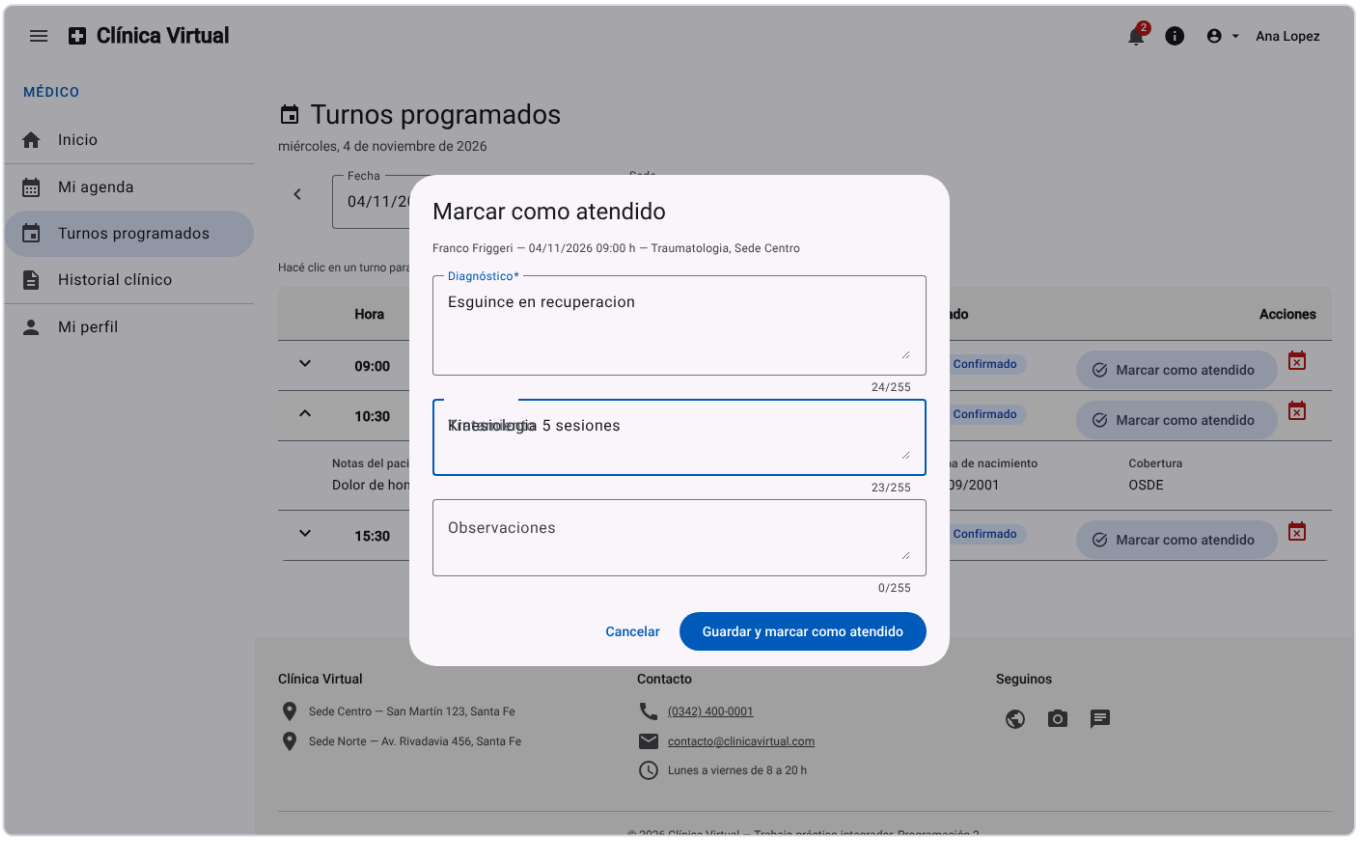
5.2 Turnos programados



Turnos del día en todas sus sedes, con filtro de sede. Al hacer clic se despliegan las notas del paciente.

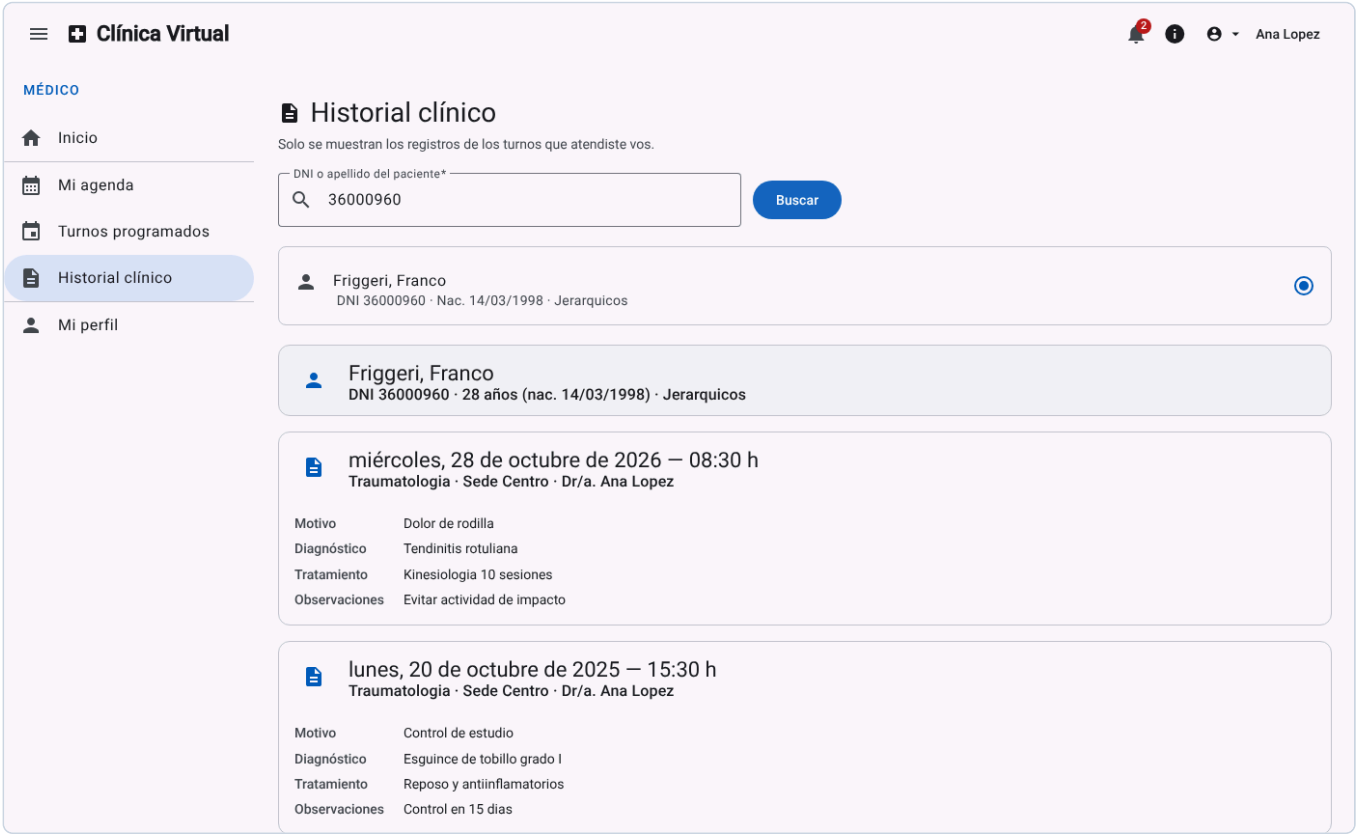
1. `GET /turnos/medico?fecha=&id_sede=` trae los turnos con los datos del paciente.

2. La tabla muestra hora, paciente, edad (calculada con la fecha de nacimiento), sede, especialidad y estado, ordenada por proximidad.
3. "Marcar como atendido" abre el diálogo para cargar diagnóstico (obligatorio), tratamiento y observaciones.
4. "Cancelar" (ícono rojo) pide confirmación.



Diálogo "Marcar como atendido": guarda el estado y el historial clínico en un solo paso.

5.3 Historial clínico

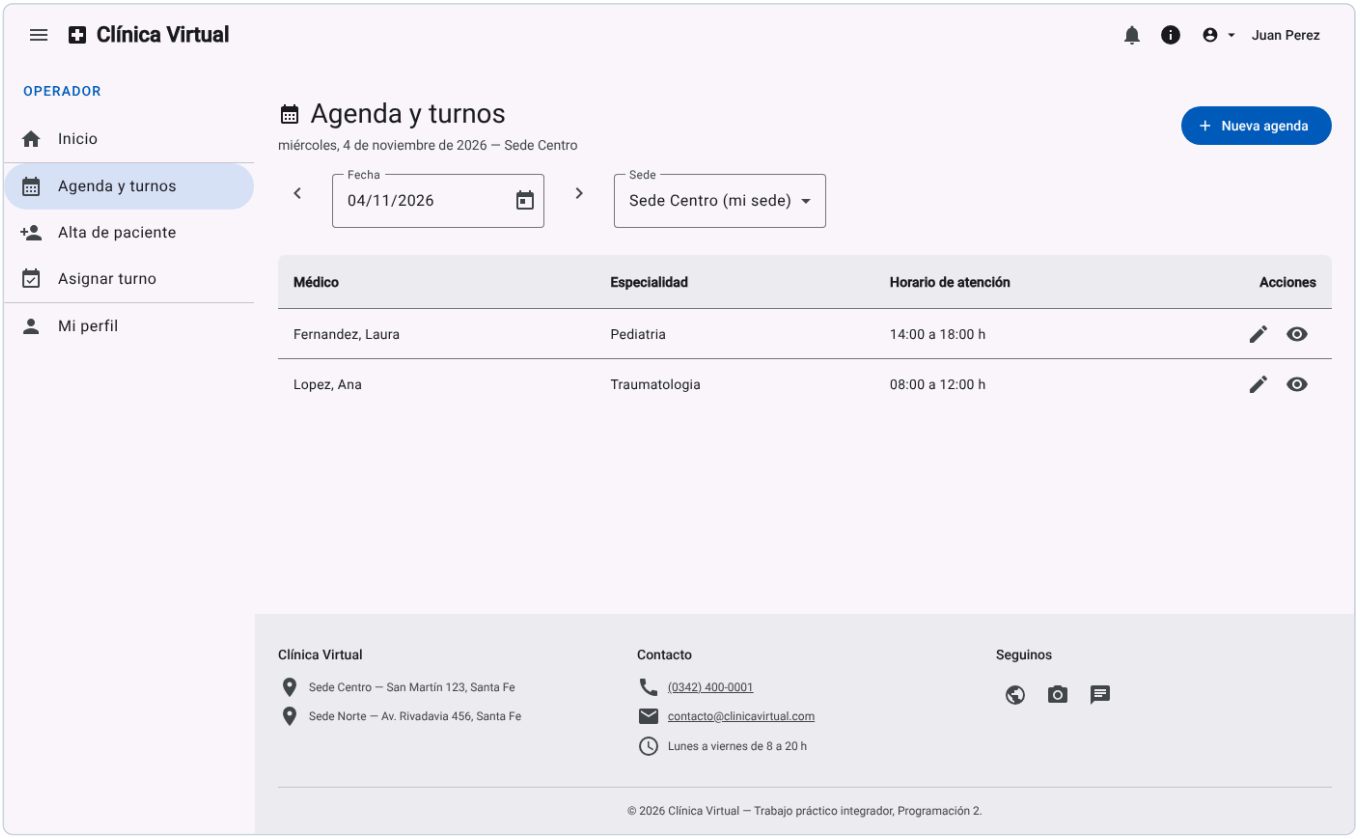


Búsqueda por DNI o apellido. Solo aparecen los registros de los turnos que atendió este médico.

> **Endpoints que usa:** `GET /pacientes?q=` · `GET /historial/:idPaciente`

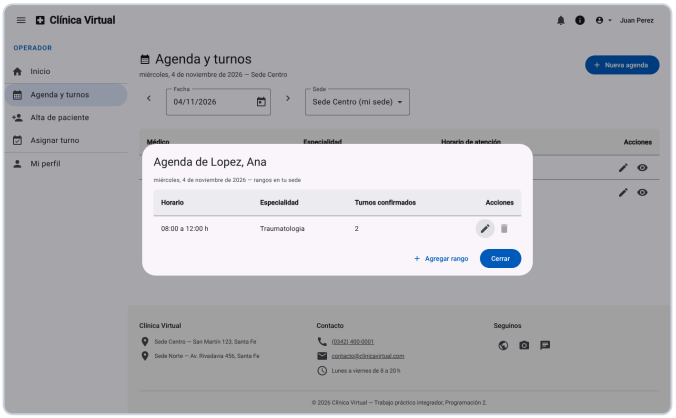
6. Flujos del operador

6.1 Agenda y turnos de la sede

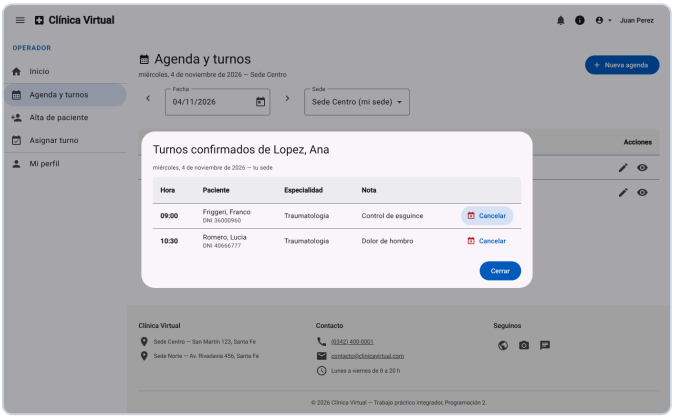


Al entrar: fecha de hoy y su sede. Una fila por médico con agenda, con el lápiz (editar agenda) y el ojo (ver turnos).

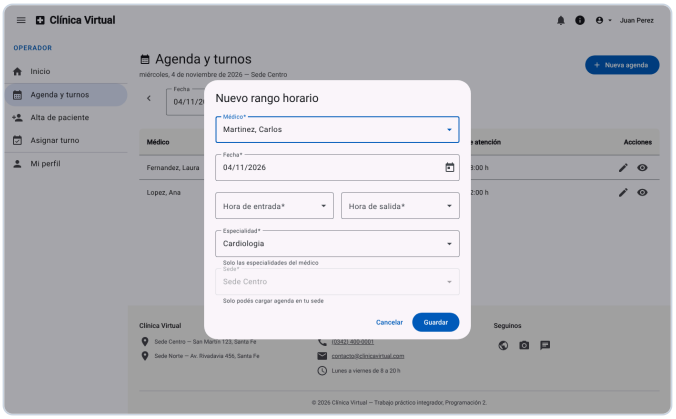
1. Filtros de fecha (hoy) y sede (la suya). Al cambiar cualquiera, la tabla se recarga (`GET /agenda?fecha&id_sede`).
2. **Lápiz**: lista los rangos del médico ese día en la sede y permite agregar, modificar o eliminar.
3. **Ojo**: turnos confirmados del médico ese día (`GET /turnos/sede?fecha&id_medico&estado=confirmado`), con "Cancelar".
4. **Nueva agenda**: se elige cualquier médico de la lista (aunque no tenga agenda) y se carga un rango; la sede queda fija en la del operador.



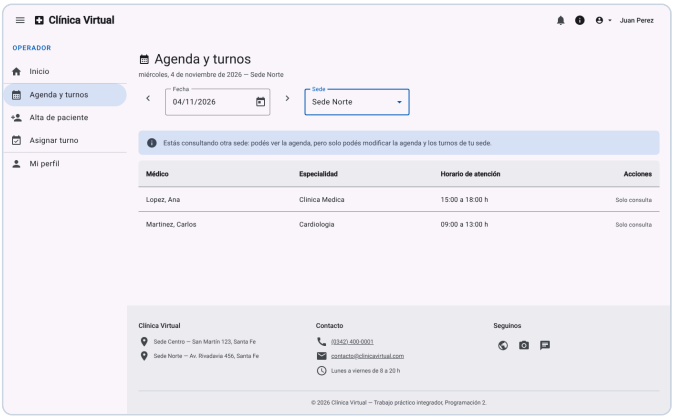
Lápiz: agenda del médico en la sede



Ojo: turnos confirmados con opción de cancelar

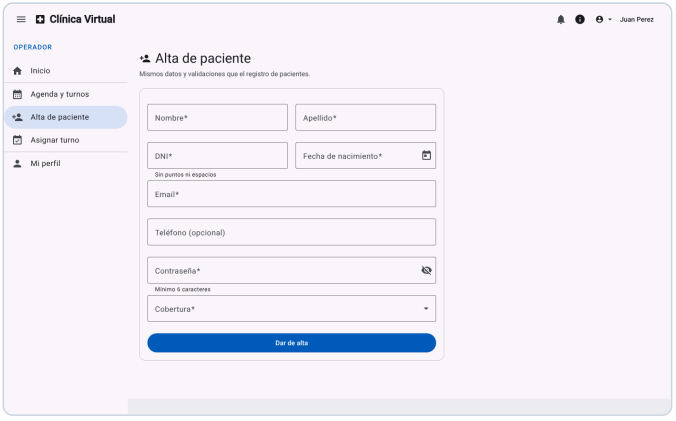


Nueva agenda: médico a elección, sede fija

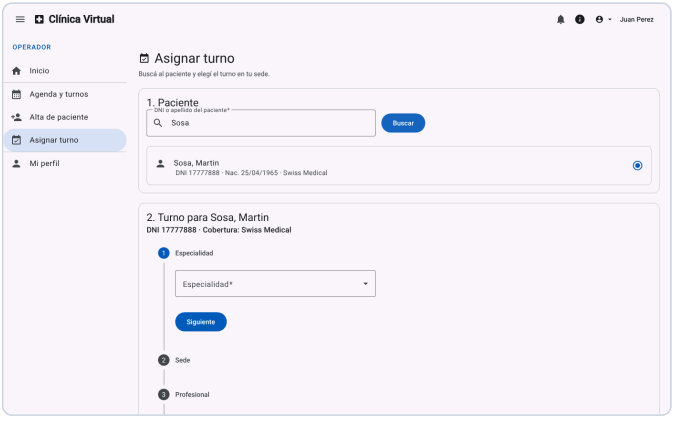


Otra sede: solo consulta, sin acciones

6.2 Alta de paciente y asignación de turno



Alta de paciente: mismo formulario que el registro



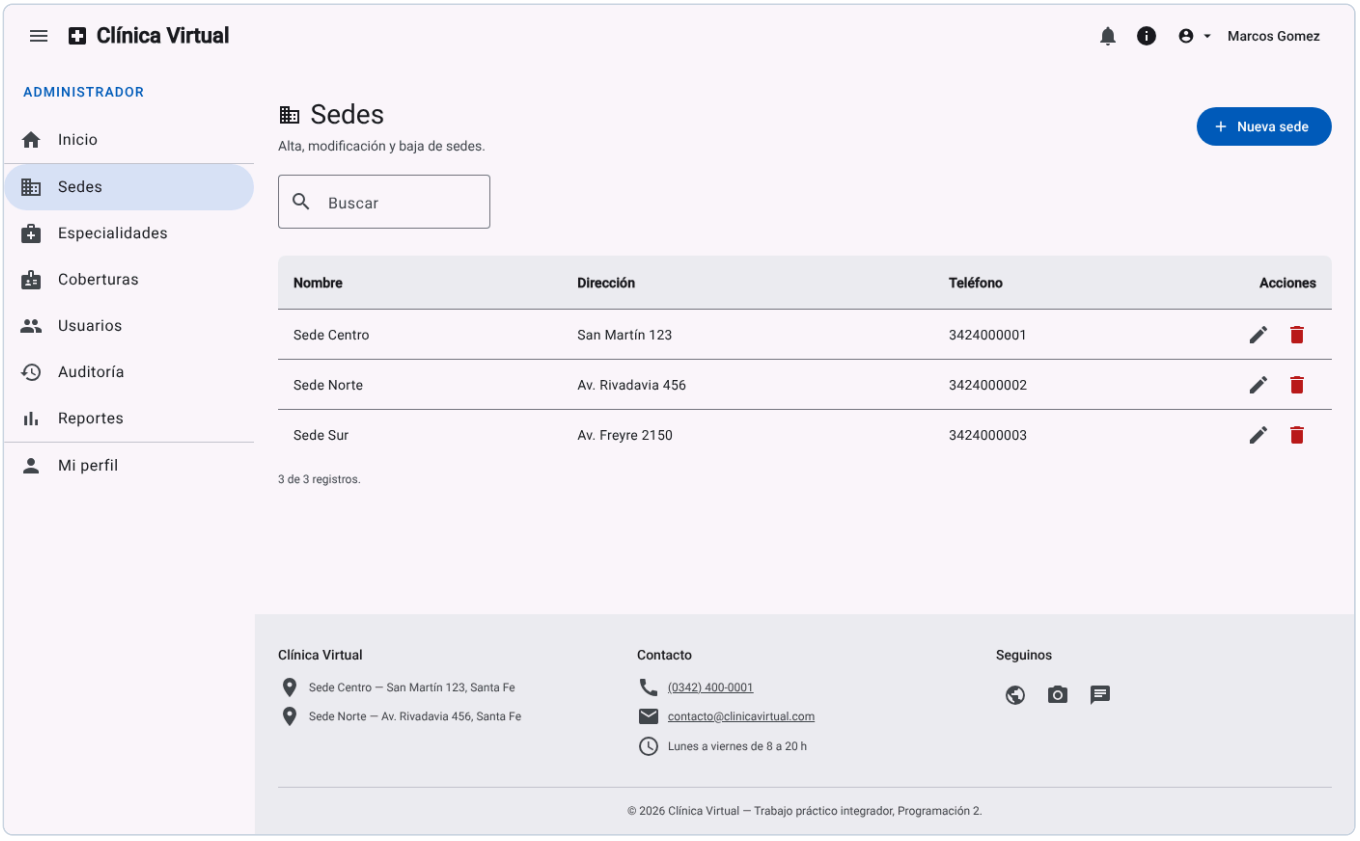
Asignar turno: buscar paciente y usar el mismo asistente

1. Alta: mismos datos y validaciones que el registro (`POST /auth/registro`, que registra al operador en la auditoría). Al terminar ofrece "Asignarle un turno".
2. Asignar: se busca al paciente por DNI o apellido y se usa el asistente de turnos con la sede fija en la del operador; el turno se crea con `id_paciente`.

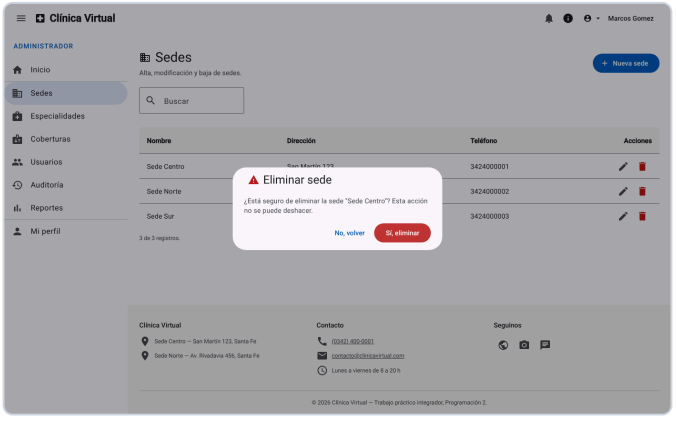
7. Flujos del administrador

7.1 Sedes, especialidades y coberturas

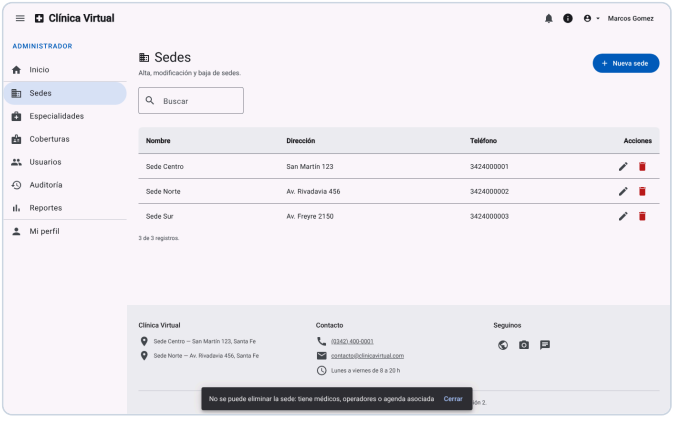
Las tres pantallas son el **mismo componente** (`pages/admin/abm-catalogo`) configurado desde la ruta: cambia el recurso, el título y los campos.



Listado con filtro de texto, alta, modificación y baja

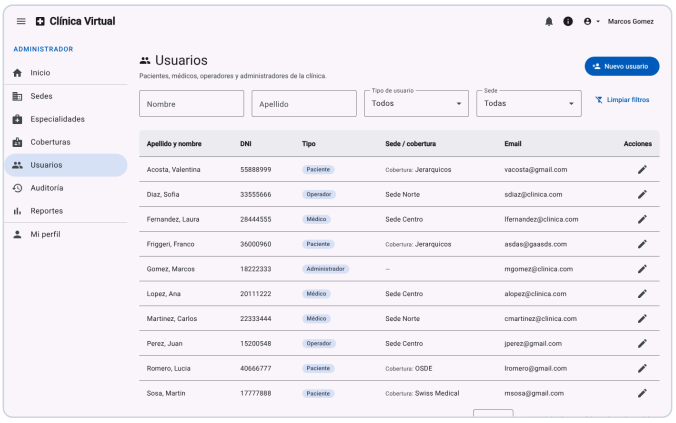


"¿Está seguro de eliminar...?"

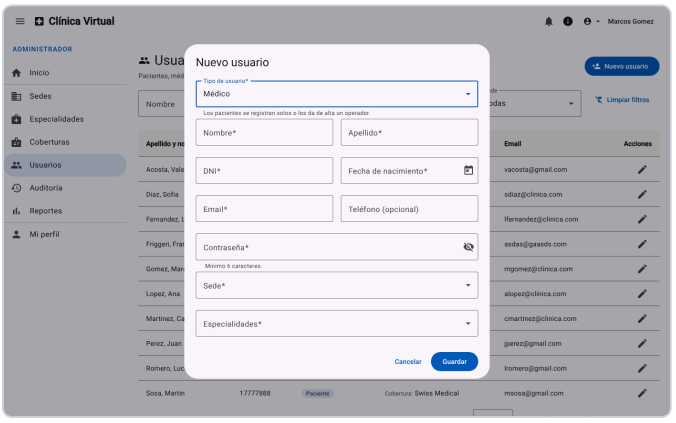


Si tiene dependencias, se muestra el mensaje del backend y el listado no cambia

7.2 Usuarios



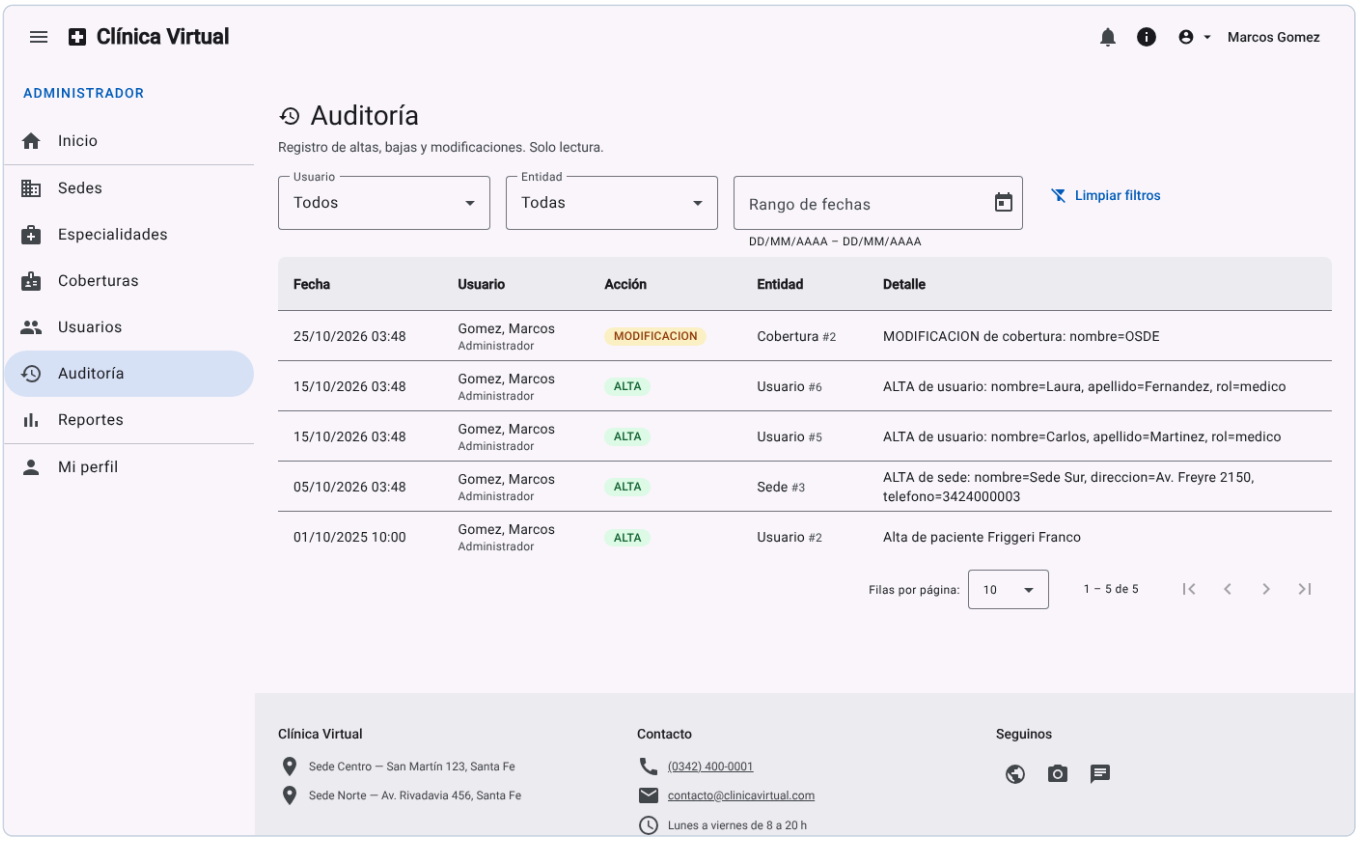
Listado paginado con filtros por nombre, apellido, tipo y sede



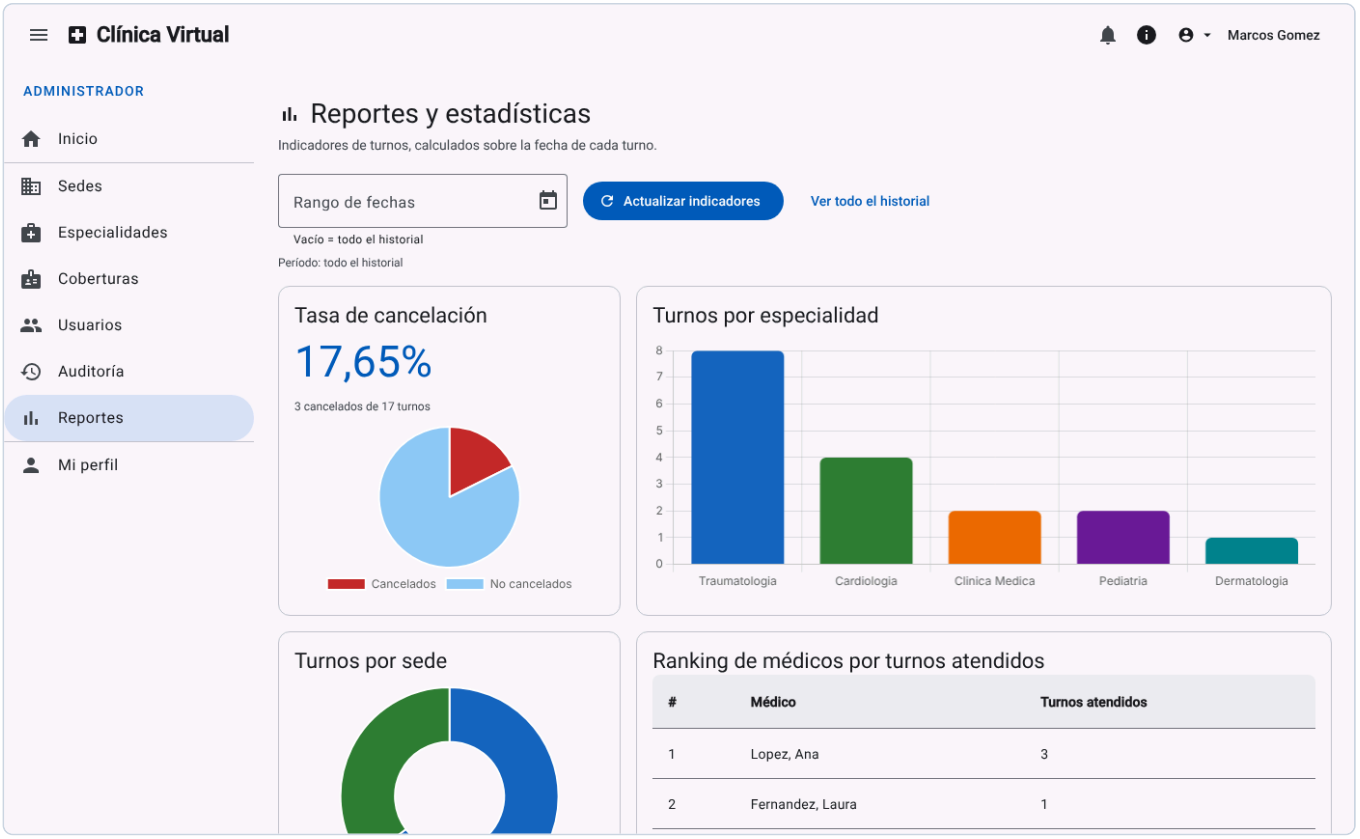
Alta: operador y médico con sede; médico con especialidades; admin sin sede

> **Endpoints que usa:** `GET /usuarios` · `GET /usuarios/:id` · `POST /usuarios` · `PUT /usuarios/:id`

7.3 Auditoría y reportes



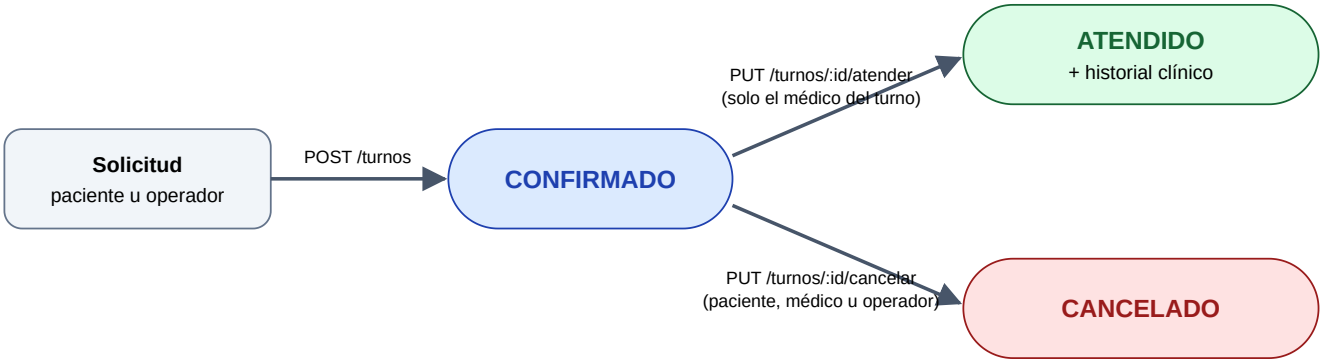
Auditoría: solo lectura, paginada, más reciente primero, filtros por usuario, entidad y fechas



Reportes: un rango de fechas actualiza los cuatro indicadores; gráficos con Chart.js

La auditoría se registra sola: el middleware `auditar(entidad)` del backend intercepta cada alta, modificación o baja exitosa de usuarios, sedes, especialidades, coberturas y agenda, y guarda quién, qué acción, sobre qué entidad y un detalle.

8. Flujo de punta a punta: la vida de un turno



Momento	Quién	Qué pasa en el sistema	Notificaciones
1. El médico (u operador) carga agenda	Médico / operador	Se crea un rango en `agenda` (fecha, horario, especialidad, sede).	—
2. Se pide el turno	Paciente / operador	La disponibilidad divide la agenda en turnos de 30 min y descuenta los reservados. `POST /turnos` crea el turno **confirmado** con la cobertura del paciente.	Paciente, médico y operador (si lo asignó)
3a. Se atiende	Médico del turno	Pasa a **atendido** y se guarda el historial clínico (diagnóstico, tratamiento, observaciones).	Paciente y médico
3b. Se cancela	Paciente, médico u operador de la sede	Pasa a **cancelado**; el horario vuelve a quedar libre.	Paciente, médico y quien canceló
4. Consulta	Paciente / médico / admin	El paciente ve el historial en "Mis turnos"; el médico en "Historial clínico"; el admin lo ve reflejado en los reportes.	—

Ejemplo para la demo

1. Lucía (paciente, 40666777) pide Traumatología → Sede Centro → Ana López. Su campana suma una notificación.
2. Ana (médica, 20111222) entra: tiene la notificación del turno nuevo y lo ve en Turnos programados.
3. Ana lo marca como atendido con un diagnóstico.
4. Lucía vuelve a entrar: en Mis turnos el turno aparece verde con el historial.
5. El admin (18222333) ve en Reportes un turno atendido más para Ana.

9. Reglas de negocio y validaciones

Regla	Frontend (antes de enviar)	Backend (validación real)
La hora de salida debe ser posterior a la de entrada	El diálogo de rango lo avisa y no envía	400
Un médico no puede tener rangos superpuestos el mismo día (aunque sea en otra sede)	—	409 con el rango con el que choca
La especialidad del rango debe ser del médico	Solo se ofrecen sus especialidades	400
No se elimina un rango con turnos confirmados	Botón eliminar deshabilitado con explicación	409
No se modifica un rango dejando turnos confirmados afuera	Aviso en el formulario	409 con los turnos afectados
El operador solo modifica agenda y turnos de su sede	Las acciones solo aparecen en su sede; sede fija en los formularios	403
El médico solo opera sobre sus turnos (en cualquier sede)	Solo ve los suyos	403
Solo se piden turnos en horarios libres de la agenda	Solo se ofrecen fechas y horarios disponibles	400 fuera de agenda / 409 ocupado
El paciente no puede tener dos turnos a la misma hora	—	409
La cobertura del turno es la del paciente	No se elige, se muestra en el resumen	Se toma de `usuario.id_cobertura`
El paciente solo edita email, contraseña, teléfono y cobertura	El resto se muestra en modo lectura	Ignora los demás campos
Operador y médico tienen sede; el admin no; el médico tiene especialidades	El formulario cambia según el tipo	400 si faltan
No se borra una sede/especialidad/cobertura con dependencias	Confirmación previa y muestra el mensaje	409
DNI de 7-8 dígitos, email válido, teléfono 6-10 dígitos, contraseña mín. 6	mat-error en cada campo	400

10. Organización de carpetas

Frontend — repositorio `santiagoaylagas-git/frontend` (rama `main`)

```
frontend/   (repositorio)
├── docs/                                      Guía de instalación y guía de flujos (PDF)
├── angular.json, package.json, tsconfig*.json   Configuración de Angular, dependencias y TypeScript
├── README.md                                  Instrucciones para levantar y probar
├── public/                                    Archivos estáticos (favicon)
└── src/
    ├── index.html, main.ts, styles.scss       Página base, arranque y estilos globales/compartidos
    ├── environments/                          URL de la API (única definición)
    └── app/
        ├── app.ts / app.html / app.scss       Componente raíz: encabezado + menú lateral + contenido + pie
        ├── app.routes.ts                      Todas las rutas, agrupadas por rol y protegidas con guards
        ├── app.config.ts                      Providers: HTTP + interceptor, idioma es-AR, datepicker, paginador
        ├── core/            Lo "no visual" que usa toda la app
        │   ├── models/      Interfaces TypeScript de lo que devuelve la API
        │   ├── services/    Un servicio por recurso de la API (llamadas HTTP)
        │   ├── interceptors/ tokenInterceptor: agrega el JWT y maneja el 401
        │   └── guards/      sesionGuard (hay sesión) y rolGuard (rol correcto)
        ├── shared/          Piezas reutilizables entre pantallas y roles
        │   ├── components/  Formularios, diálogos y avisos compartidos
        │   ├── pipes/       fecha: muestra DD/MM/AAAA
        │   └── utils/       Funciones de fechas, adaptador del datepicker, paginador en castellano
        ├── layout/          Encabezado (campana), menú lateral y pie de página
        └── pages/           Una carpeta por pantalla
            ├── paciente/  medico/  operador/  admin/
            └── inicio, login, registro, perfil, acceso-denegado, no-encontrada
```

Carpeta / archivo	Qué guarda
`core/models/`	`usuario`, `catalogo` (sede, especialidad, médico), `agenda`, `turno` (turno, historial, disponibilidad), `paciente`, `notificacion`, `auditoria`, `reporte`, `respuesta` (estructura uniforme).
`core/services/api.ts`	`API_URL` y helpers: `datos()` extrae `datos` de la respuesta; `parametros()` arma la query sin vacíos.
`core/services/*.service.ts`	`auth` (login, perfil, registro, vencimiento), `catalogo` (sedes, especialidades, coberturas), `medico`, `agenda`, `turno` (turnos y disponibilidad), `paciente` (búsqueda e historial), `usuario`, `auditoria`, `reporte`, `notificacion` (estado de la campana con signals), `errores` (traduce errores HTTP a mensajes).
`shared/components/formulario-paciente`	Alta de paciente: lo usan el registro y el operador.
`shared/components/solicitud-turno`	Asistente de turno de 6 pasos: lo usan el paciente y el operador.
`shared/components/rango-dialogo`	Alta/modificación de rangos de agenda: médico y operador.
`shared/components/buscador-pacientes`	Búsqueda por DNI/apellido: médico (historial) y operador (asignar turno).
`shared/components/lista-historial`	Tarjetas del historial clínico: paciente y médico.
`shared/components/aviso, cargando, confirmar-dialogo, estado-turno`	Mensajes de error/vacío, indicador de carga, "¿Está seguro?", etiqueta de color del estado del turno.
`pages/paciente/`	`mis-turnos`, `solicitar-turno`, `mi-historial`, `mis-datos`.
`pages/medico/`	`mi-agenda`, `turnos-programados` (+ `atender-dialogo`), `historial-clinico`.
`pages/operador/`	`agenda-sede` (+ `agenda-medico-dialogo` y `turnos-medico-dialogo`), `alta-paciente`, `asignar-turno`.
`pages/admin/`	`abm-catalogo` (+ `abm-dialogo`, sirve para sedes, especialidades y coberturas), `usuarios` (+ `usuario-dialogo`), `auditoria`, `reportes`.

Cada pantalla es un componente *standalone* con su `.ts` (lógica), `.html` (plantilla) y a veces `.scss` (estilos); las más chicas tienen la plantilla dentro del `.ts`.

Backend — repositorio `santiagoaylagas-git/backend`

```
backend/   (repositorio)
├── README.md                Documentación: ramas, carpetas, endpoints y cambios
├── package.json             Dependencias y scripts (dev, start, db:importar)
├── .env.example             Variables de entorno de ejemplo (copiar a .env)
├── .vscode/                 Tareas y depuración para VS Code / Antigravity
├── database/
│   └── clinica_ampliada.sql.txt   Tablas + datos de prueba (fechas relativas a hoy)
├── postman/                 Colección de Postman (112 requests con tests)
├── scripts/                 importar-db.js (multiplataforma) e importar-db.ps1 (XAMPP)
└── src/
    ├── server.js            Arranca el servidor en el puerto del .env
    ├── app.js               Express: CORS, JSON, rutas, 404 y manejador global de errores
    ├── database/db.js       Pool de conexiones a MySQL
    ├── routes/              Qué URL va a qué controlador y con qué protección
    ├── middlewares/         auth.js (token y rol) y auditoria.js (registro automático)
    ├── controllers/         La lógica de cada recurso
    └── utils/               respuesta.js, fechas.js, notificaciones.js
```

Controlador	Qué resuelve
`authController`	Registro, login, perfil y actualización de datos del paciente.
`sedeController`, `especialidadController`, `coberturaController`	ABM de catálogos con control de dependencias.
`usuarioController`	Listado paginado, alta y edición de usuarios (con especialidades del médico).
`medicoController`	Listado de médicos con especialidades, filtrable.
`agendaController`	Rangos horarios con todas las validaciones de sede, superposición y turnos confirmados.
`disponibilidadController`	Fechas y horarios libres para pedir turno.
`turnoController`	Alta, cancelación, atención (con historial) y listados por paciente, médico y sede.
`historialController`	Carga y consulta del historial clínico.
`pacienteController`	Búsqueda de pacientes por DNI o apellido.
`notificacionController`	Listar y marcar como leídas.
`auditoriaController`, `reporteController`	Log de auditoría (paginado y filtros) y los cuatro indicadores.
`healthController`	`GET /health` para verificar la conexión a la base.

Tablas de la base de datos

Tabla	Guarda
`usuario`	Todos los usuarios (los 4 roles), con sede o cobertura según corresponda; contraseña con bcrypt.
`sede`, `especialidad`, `cobertura`	Catálogos.
`medico_especialidad`	Qué especialidades tiene cada médico.
`agenda`	Rangos horarios: médico, fecha, entrada, salida, especialidad y sede.
`turno`	Turnos: rango de agenda, fecha, hora, paciente, cobertura, nota y estado.
`historial_clinico`	Diagnóstico, tratamiento y observaciones de cada turno atendido.
`notificacion`	Mensajes para cada usuario, leídos o no.
`log_auditoria`	Quién hizo qué alta, baja o modificación y cuándo.

11. Cómo levantar el sistema y usuarios de prueba

1. Iniciar MySQL (por ejemplo con XAMPP).
2. Backend: `cd backend` → `npm install` → copiar `.env.example` a `.env` (cambiar `JWT_SECRET`) → `npm run db:importar` → `npm run dev` (queda en `http://localhost:3000`).
3. Frontend: `cd frontend` → `npm install` → `npm start` → abrir `http://localhost:4200`.
4. Reimportar la base el día de la defensa para tener agenda y turnos de hoy.

Contraseña de todos: **test1234**.

Rol	DNI	Nombre	Datos útiles
Administrador	18222333	Marcos Gómez	Sin sede
Médico	20111222	Ana López	Traumatología y Clínica Médica · Sede Centro y Sede Norte
Médico	22333444	Carlos Martínez	Cardiología · Sede Norte
Médico	28444555	Laura Fernández	Pediatría y Dermatología · Sede Centro
Operador	15200548	Juan Pérez	Sede Centro
Operador	33555666	Sofía Díaz	Sede Norte
Paciente	36000960	Franco Friggeri	Turnos en los tres estados e historial
Paciente	40666777	Lucía Romero	OSDE
Paciente	17777888	Martín Sosa	Swiss Medical
Paciente	55888999	Valentina Acosta	Jerárquicos